

Town Interaction Feature Demo

Overview

The Town feature provides a social hub and shared space for Koko companions. It connects single-player pet interactions with lightweight co-op presence: invites, shared town maps, nearby QR join, and a virtual friend device for testing. The town is meant to be a low-friction social surface that augments the pet experience without introducing heavy real-time complexity.

Goals

- Let players see other Koko instances on a shared map and feel presence without heavy real-time requirements.
- Make invites and sharing simple (share link, QR scan, invite code persistence).
- Provide robust fallbacks: local invite state when cloud is unavailable, and a login-based fallback for environments missing dedicated cloud functions.
- Keep the UI light and mobile-friendly: short micro-interactions, clear affordances, and graceful degradation.

Key User Flows

1. Create invite: the owner taps "Co-op / Create invite" to generate an invite code and, where possible, a QR code. The server stores invite meta on the owner profile and returns an invite path and QR file reference.
2. Share & join: inviter shares link or QR code. Another user joins by visiting the invite or by using the code; the cloud updates room membership and presence.
3. Presence & heartbeat: each client periodically sends its presence (x,y,pet action, avatar, pet name) to the town cloud function. The cloud persists presence under the room owner doc so other members can load consistent state.
4. Offline handling: when a user disconnects or marks offline, their last known position and lastSeenAt are preserved and displayed as offline. The UI differentiates online vs offline partners.
5. Virtual friend: a local virtual friend device can be connected to emulate a second user, enabling testing of the co-op experience without a second phone.

UI & Interaction Notes

- Map and hotspots: the town map displays building hotspots (home, playground, shop, task-board, talk-house). Tapping hotspots opens place cards or transitions to pages (shop, play, tasks, chat).
- Pet movement: tapping the map moves Koko to the tapped coordinate with an animated walk and simple easing. Pet position and action are stored and sent as presence.
- Community panel: a co-op panel shows partners, online status, join/copy/share controls, and a QR preview when available.
- Guide & onboarding: an interactive town guide introduces map, pet movement, and co-op. The guide can be dismissed with a visible close control.
- Visual layering: pets and UI panels are layered to avoid clipping; pets behind panels are unmounted when the co-op panel overlays the map to prevent visual conflicts.

Technical Design

Cloud contract

- Cloud function `town-community` accepts actions: `load`, `heartbeat`, `createInvite`, `joinInvite`, `offline`.
- Owner user documents hold `townInviteCode`, `townInviteExpiresAt`, `townQrCodeFileID`, `townRoomOwnerOpenid`, `townMemberOpenids`, and a `townPresence` object.
- The cloud returns a normalized `TownCommunityState` containing `room`, `partners`, `invitePath`, `inviteCode`, and optionally `qrCodeFileID`.

Presence model

- Presence payload includes `nickName`, `avatarUrl`, `petName`, `x`, `y`, `action`, and `online`.
- The cloud computes online via a TTL: presence is considered online if `lastSeenAt` is within the configured window (e.g. 90s).
- Client-side heartbeat calls `sendTownCommunityHeartbeat(payload)` on a regular interval and reacts to the returned state.

Fallbacks and resilience

- Primary call path: `wx.cloud.callFunction({ name: 'town-community' })`.
- Fallback path: when `town-community` is missing or returns errors likely caused by missing cloud function, calls are routed to the `login` cloud function with `action: 'townCommunity'` to preserve basic behavior.

- Local fallback: if cloud calls fail and the action does not strictly require server-side creation (e.g. transient load), clients create a local invite/state and continue the session without server functionality. Local invite codes are persisted to storage for the session.

Invite & QR

- Invite codes are generated server-side (6-character alphanumeric) and have an expiration (e.g. 14 days). The server attempts to generate and upload a QR code image using `cloud.openapi.wxacode.getUnlimited`; if QR creation fails the invite still works via link/code.
- The client persists invite codes to local storage and uses them when appropriate (nested invite param handling, restoring pending invites after login).

Iteration Summary (selected highlights)

This section summarizes the main iterations implemented while building the Town feature (commit labels used as shorthand):

- Virtual friend device: `Add virtual town co-op friend` — added a local virtual friend to test co-op without a second phone.
- Invite sync fixes: `Fix town community invite sync` — corrected server/client flows when invite was created, refreshed, or nested in URLs.
- Preserve invites through login: `Preserve town invites through login` — ensured invites survive login redirects and guest → wechat transitions.
- Login fallback: `Fallback town co-op through login cloud function` — added a fallback call path into the `login` function when `town-community` isn't deployed.
- Cloud presence robustness: `Keep town co-op usable when cloud state is empty` — handle empty or missing cloud state by creating local session state.
- Capacity & profile storage: `Show town co-op friend capacity and Store town co-op state on user profiles` — expose capacity in the HUD and persist room/member references in user documents.
- Sharing UX: `Keep invite share actions enabled and Enable sharing when invite code is nested` — keep sharing controls active across deep-link or nested invite cases.
- Guide and layering fixes: `Add town guide close button, Unmount town pets behind co-op panel, Keep town dog below co-op panel` — refine onboarding guide and visual stacking to prevent clipping and interaction issues.

Implementation Notes and Gotchas

- Map images: the client tries multiple map sources (webp/jpg) and falls back gracefully; unreadable assets switch to a fallback candidate automatically.
- Coordinate clamping: pet and partner coordinates are clamped to visible bounds to avoid off-map positions.
- Older runtimes: some cloud and client behaviors include fallbacks to support older runtime versions or different WeChat environments.
- Limitations: the current design favors a low-member-count room (`MAX_ROOM_MEMBERS = 2`). The server logic and UI assume small groups rather than massive rooms.

Next Steps and Recommendations

- Expand room capacity and test scaling behaviors (more than 2 members).
- Improve QR availability by queuing QR generation and exposing a retry UI when `getUnlimited` fails.
- Add richer co-op interactions (e.g. short shared tasks or synchronized events) while keeping the heartbeat model lightweight.
- Add analytics events around invite generation, join rates, and virtual friend usage to inform future UX improvements.
- Validate offline-first behaviors on slower networks and across WeChat DevTools scenarios.

File references

- Town page implementation: `src/pages/TownPage.vue`
- Client side town cloud service: `src/services/townCommunityCloud.ts`
- Cloud function: `cloudfunctions/town-community/index.js`

Generated from the current repository: summarizes design, flows, cloud contract, resilience strategy, and iteration history for the Town co-op feature.